

The background is a blurred image of a computer monitor. It shows a dark-themed code editor with various lines of code in different colors (blue, green, white). At the bottom, a taskbar is visible with several application icons, including what looks like a web browser, a file explorer, and some development tools. The overall image is out of focus, serving as a backdrop for the text.

C#

Level 1

Escape Characters



C#

Level 1

Escape Characters

Definition

- ◆ Escape characters in *C#* are special character sequences that begin with a backslash ``\`` and are used to represent characters or actions that are difficult or impossible to type directly inside a string.
- ◆ They are commonly used with strings such as:
`Console.WriteLine("Text");`

Explanation

- ◆ An escape sequence usually starts with:

\

- ◆ The character after ` \ ` determines what action or character will be produced.

Common Escape Characters

 ` \n ` → New line

 ` \t ` → Tab

 ` \b ` → Backspace

 ` \' ` → Single quote

 ` \" ` → Double quote

 ` \\ ` → Backslash

 ` \a ` → Alert / Beep

☐ Procedures

☐ Newline ` \n`

- ◆ ` \n` moves the cursor to a ****new line****.

```
Console.WriteLine("Welcome to \nProgrammingAdvices");
```

2☐ Tab ` \t`

- ◆ ` \t` inserts a ****tab space****.

```
Console.WriteLine("Welcome to\tProgrammingAdvices");
```

3☐ Backspace ` \b`

- ◆ ` \b` moves the cursor one position backward.

```
Console.WriteLine("Welcome to \bProgrammingAdvices");
```

- ◆ Its exact visible effect can depend on the console environment.

4☐ Single Quote ` \'`

- ◆ ` \'` allows you to include a single quote character inside a string.

```
Console.WriteLine("Welcome to \'ProgrammingAdvices");
```

5☐ Double Quote ` \"`

- ◆ ` \"` allows you to include a double quote inside a double-quoted string.

```
Console.WriteLine("Welcome to \"ProgrammingAdvices\"");
```

6☐ Backslash ` \\`

- ◆ ` \\` represents a single backslash ` \`.

```
Console.WriteLine("C:\\Users\\Sara");
```

7☐ Alert ` \a`

- ◆ ` \a` represents the ****alert/bell character****.

```
Console.WriteLine("\a");
```

💡 Example

```
Console.WriteLine("Useful Escape Characters:\n");  
//Newline  
Console.WriteLine("Newline:");  
Console.WriteLine("Welcome to \nC#\n");  
//Tab  
Console.WriteLine("Tab:");  
Console.WriteLine("Welcome to \tC#\n");  
//Backspace  
Console.WriteLine("Backspace:");  
Console.WriteLine("Welcome to \bC#\n");  
//Single quote  
Console.WriteLine("Single Quote:");  
Console.WriteLine("Welcome to \'C#\n");  
//Double quote  
Console.WriteLine("Double Quote:");  
Console.WriteLine("Welcome to \"C#\n");  
//Backslash  
Console.WriteLine("Backslash:");  
Console.WriteLine("Welcome to \\ C#\n");  
//Alert  
Console.WriteLine("Alert:"); Console.WriteLine("\a");
```

Note

- ◆ Escape characters are interpreted ****inside regular C# string literals****.
- ◆ For example:
 `Console.WriteLine("Hello\nWorld");`
 is different from:
 `Console.WriteLine(@"Hello\nWorld");`
- ◆ The ``@`` creates a ****verbatim string****, where ``\n`` is treated as ordinary text rather than a newline.

⊘ Warning

⚠ A single backslash ` \ ` has a special meaning inside a regular string.

✗ Incorrect:

```
Console.WriteLine("C:\\Users\\Sara");
```

✓ Correct:

```
Console.WriteLine("C:\\\\Users\\\\Sara");
```

✓ Or use a verbatim string:

```
Console.WriteLine(@"C:\Users\Sara");
```



Comparison

▨ Normal String

`Console.WriteLine("Hello\nWorld");`

- ✓ Escape sequences are interpreted
- ✓ `` \n `` creates a new line

▤ Verbatim String

`Console.WriteLine(@"Hello\nWorld");`

- ✓ Backslashes are treated literally
- ✓ `` \n `` appears as `` \n ``

✦ Summary

- ✓ Escape characters start with ``\``
- ✓ ``\n`` → New line
- ✓ ``\t`` → Tab
- ✓ ``\b`` → Backspace
- ✓ ``\'`` → Single quote
- ✓ ``\"`` → Double quote
- ✓ ``\\`` → Backslash
- ✓ ``\a`` → Alert / Beep
- ✓ They are mainly used to control or represent special characters inside strings

Thank you 😊